

Introduction to Java

Chapter 1 Summary Notes

1. Introduction to Languages

A **language** is a way of communication between the user and the system/machine. Programming languages are classified into three broad levels:

- **High Level Language** – Languages such as Java, Python, C and C++. Written close to English so that it is easy for humans to read, write and understand.
- **Middle Level Language** – Used to communicate directly with hardware, e.g. Arduino, microprocessors and microcontrollers.
- **Low Level Language** – The machine-understandable byte code made up of binary values (0s and 1s). Characters are interpreted using their **ASCII values**, which are derived by converting a character's numeric value into its binary form.

2. Overview of Java

- Java was created by **James Gosling** at **Sun Microsystems**; the project began under the internal name "**Oak**".
- Sun Microsystems was later acquired by **Oracle**, which now maintains Java.
- Java is a fully **Object-Oriented**, **open-source**, high-level programming language.
- It is **platform-independent** ("Write Once, Run Anywhere" – WORA) because compiled code runs on any machine that has a JVM.
- Java performs automatic **Garbage Collection**, freeing up memory so that other programs/processes can use it.

The latest version referenced in the notes is **Java 26. Timeline / History of Java**

Year	Milestone
1991	Project started by James Gosling at Sun Microsystems, originally called "Oak"
1995	Renamed from Oak to Java
1996	JDK 1.0 released
1999	Java 2 released, split into editions: J2SE, J2EE, J2ME
2006	Made open source under the GPL (General Public License)
2010	Oracle acquired Sun Microsystems
2014 →	Java began following a periodic (roughly 6-month) release cycle

3. Features of Java

- **Object-Oriented** – Everything in Java is treated as an object.
- **Platform-Independent** – "Write Once, Run Anywhere" (WORA) using the JVM.
- **Simple & Easy to Learn** – Fewer complexities compared to C++.
- **Secure** – Provides runtime checking and has no explicit pointers.
- **Robust** – Strong memory management and exception handling.
- **Multithreaded** – Supports concurrent execution of multiple tasks.
- **High Performance** – Uses the Just-In-Time (JIT) compiler.
- **Distributed** – Supports distributed applications through RMI and EJB.
- **Portable** – Java bytecode can run on any machine that has a JVM.
- **Scalable** – Well suited to applications that need to grow in size and complexity.

Note: RMI (Remote Method Invocation) and EJB (Enterprise Java Beans) are frameworks that allow Java objects to communicate and work across a network

4. Compiler in Java

- Converts Java source code (.java) into **byte code** (.class file) with the help of the JVM.
- Checks the code for correct **syntax** before compilation is completed.
- The generated .class file is stored in the **bin** folder of the project.

5. Interpreter in Java

- Executes the compiled byte code, reading and running it **line by line**.
- Converts byte code into **machine code**; uses the **Just-In-Time (JIT) Compiler** to improve performance.
- Detects and reports errors line by line during execution.

6. Compilation & Execution Workflow

Java file (.java) → Compiler (javac) → Byte code (.class) → Interpreter (with JIT) → Native Machine Code (0s and 1s, understood directly by the machine)

7. JVM – Java Virtual Machine

- Loads the byte code, **verifies** it (checks syntax/security), and **executes** it with the help of the JIT compiler.
- Handles both compilation and interpretation internally.
- Runs the **garbage collector** to free unused memory during execution.
- Because every platform has its own JVM implementation, the same byte code can run anywhere — this is the basis of WORA.

8. JRE – Java Runtime Environment

- Comprises the **JVM** plus the required libraries and supporting files.
- Provides the environment needed to **run** a Java application — it is not used for compilation.
- Formula: **JRE = JVM + required libraries**.

9. JDK – Java Development Kit

- Allows a developer to **write, compile, and run** Java code — required for compiling and modifying code (JRE alone cannot compile).
- Key tools bundled with the JDK:
- **javac** (Compiler) – converts .java files to byte code.
- **jdb** (Debugger) – helps find and fix errors.
- **javadoc** (Doc generator) – generates documentation from code.

10. IDE – Integrated Development Environment

Provides a platform where developers can write, compile, debug and run code. Popular IDEs for Java include **Eclipse**, **IntelliJ IDEA**, **NetBeans** and **VS Code**.

Key features:

- Compiler integration (Java → byte code)
- Code Editor
- Interpreter integration (byte code → machine code)
- Debugger to find and fix errors
- Project management – organizing files and libraries
- Additional tools – build tools, version control (e.g. Git)

11. Java's Magic Bytecode – CAFEBAFE

- Every Java **.class** file begins with the hexadecimal signature **CAFEBAFE**, acting like an ID.
 - When the JVM opens a **.class** file, it checks whether it starts with CAFEBAFE to confirm it is valid Java byte code.
- If the magic number is missing, the JVM **rejects** the file; this check also helps ensure the file is safe to execute.

12. JIT – Just-In-Time Compiler

- Works alongside the interpreter to make execution **faster**.
- Especially useful for large data sets or code with many loops/repeated method calls.
- Compiles frequently-used parts of the byte code (e.g. loops) directly into **native code**, so it can be run directly instead of being interpreted again and again.

13. Native Machine Code

- The final form of executable instructions, made up purely of **binary (0s and 1s)**.
- Directly understandable by the machine's hardware.
- Overall path: **Source code (.java) → Compiler (javac) → Byte code (.class) → JVM (Interpreter + JIT) → Native Machine Code**.

14. Related Topics Noted for Further Study

- **Design approaches:** Top-down vs. Bottom-up program design.
- **Frameworks built on Java:** Spring Boot (backend); also referenced alongside React on the frontend side of full-stack development.
- **Typical web-app layers:** HTML – Skeleton/structure, CSS – Design/beauty, Back-end – Logic/"life" of the application.
- **Open questions raised in the notes:** "Why Java?", which everyday/regular applications use Java vs. Python, and what RMI & EJB are in more depth.